MODULE 28 · EXAM STRATEGY - ALL DOMAINS

# Confusion Pairs

The services and features the exam sets against each other, sorted by the one question that separates each pair.

DOMAIN All six  EXAM WEIGHT 100%  MODULE 28 of 34

admin@sthithapragna.com

Every confusion pair differs along **one axis**. Find the axis, ask the question, and the pair splits.

# Axis 1: before, or after?

| PAIR | BEFORE | AFTER |
|---|---|---|
| Hook / Config rule | Hook blocks the stack | Rule reports the resource |
| Proactive / detective control | Stops creation | Finds non-compliance |
| Deployment / stage rollback | During the traffic shift | When the stage completes |
| Change set / drift detection | Previews your change | Finds someone else's |
| Policy check / Access Analyzer | In the pipeline | In the running account |

ON THE EXAM

'Ensure' and 'prevent' want the left column. 'Identify' and 'report' want the right.

# Axis 2: who, or what, is acting?

| PAIR | FIRST | SECOND |
|---|---|---|
| SCP / RCP | Your principals | Resources, against outsiders |
| Execution / task role | ECS, before code runs | Your application |
| Action / CFN execution role | Starts the operation | Creates the resources |
| Instance profile / service role | Agent fetches the bundle | CodeDeploy manages ASG, ELB |
| ExternalId / aws:SourceArn | A third-party vendor | An AWS service |

ON THE EXAM

Access denied: name who is actually making the call before touching any policy.

# Axis 3: what kind of problem?

| PAIR | FIRST | SECOND |
| --- | --- | --- |
| Macie / GuardDuty S3 Protection | What is in the objects | How they are accessed |
| GuardDuty / Inspector | Active threats | Vulnerable software |
| CloudWatch investigations / Detective | Operational root cause | Security incident scope |
| Config / CloudTrail | What the resource looked like | Who called which API |
| Security Hub CSPM / new Security Hub | Posture against standards | Correlated exposure |

ON THE EXAM

Read the noun in the requirement: data, threat, vulnerability, outage, configuration, or caller.

# Axis 4: copy it, or look at it?

| PAIR | COPY | LOOK |
| --- | --- | --- |
| Log centralisation / cross-account observability | Retained copy, cross-Region | View in place, same Region |
| ECR replication / pull through cache | Every push, everywhere | Only what is pulled |
| Conformance pack / aggregator | Deploys the rules | Sees the results |
| Backup copy / restore testing | Protects the data | Proves the data restores |
| Organisation trail / Detective | Records everything | Explains one principal |

ON THE EXAM

'Retain for years' means copy. 'Operators need to see' means look.

# Axis 5: once, continuously, or on a schedule?

| PAIR | FIRST | SECOND |
|---|---|---|
| Run Command / State Manager | Once, now | Continuously, including new nodes |
| Export task / subscription filter | Batch | Real time |
| ECR basic / enhanced scanning | On push | Continuous rescans |
| Scheduled rule / Scheduler | UTC cron, recurring | Time zones, one-off runs |
| Config continuous / periodic recording | Every change | Once a day, cheaper |

ON THE EXAM

'Including new instances' and 'as new vulnerabilities appear' both mean continuous.

# Axis 6: how much is already running?

| PAIR | LESS RUNNING | MORE RUNNING |
|---|---|---|
| Pilot light / warm standby | Data live, compute off | Small working copy |
| Multi-AZ standby / read replica | Failover only, not readable | Serves reads, manual promote |
| Zonal shift / Region failover | One bad zone | A whole Region lost |
| Predictive scaling / warm pool | Launches ahead of a pattern | Pre-initialised, for slow starts |
| Reserved / provisioned concurrency | A guaranteed share | Pre-warmed environments |

ON THE EXAM

Tight recovery or start-up times need more already running. Pay only for what the requirement demands.

# Axis 7: where does the failure land?

| PAIR | FIRST | SECOND |
|---|---|---|
| Target DLQ / archive and replay | Could not be delivered | Delivered, then mishandled |
| DLQ / on-failure destination | The event alone | The event plus the error |
| Circuit breaker / deployment alarms | Tasks that will not run | Tasks that run badly |
| notBreaching / breaching | Silence is good | Silence is bad |
| Retry / rollback | Same code, transient fault | Bad code, go back |

ON THE EXAM

Ask what actually went wrong: delivery, processing, starting, running, or the code itself.

# Axis 8: which layer, which key, which store?

| PAIR | FIRST | SECOND |
|---|---|---|
| WAF / Network Firewall | Attacks inside HTTP | Traffic by domain or port |
| Security group / network ACL | Stateful, allow only | Stateless, can deny |
| Gateway / interface endpoint | S3 and DynamoDB, free | Most services, charged |
| AWS managed / customer managed key | Cannot be shared | Cross-account, your policy |
| Secrets Manager / Parameter Store | Rotation built in | Configuration, free tier |

ON THE EXAM

Cross-account anything with encryption: customer managed key. Rotation required: Secrets Manager.

> An RDS for MySQL database uses a Multi-AZ DB instance deployment. Reporting queries saturate the primary's CPU during business hours.
>
> The team expected the standby to absorb those reads, but nothing changed. The database must keep automatic failover.

## What should the DevOps engineer do?

A Resize the standby to a larger instance class

B Add a read replica, point the reporting queries at it, and keep the Multi-AZ deployment

C Replace Multi-AZ with a read replica in another Availability Zone

D Put RDS Proxy in front of the database

◆ PAUSE AND DECIDE

## What should the DevOps engineer do?

A **Bigger standby**

*A Multi-AZ instance standby serves no reads, whatever its size.*

B **Read replica plus Multi-AZ**

The replica takes reporting reads; Multi-AZ keeps automatic failover.

C **Replica instead of Multi-AZ**

*Loses automatic failover, which is required.*

D **RDS Proxy**

*Pools connections. The problem is CPU from queries.*

THE PAIR

Axis 6: a standby is for failover, a replica is for reads. Most production databases want both.

> A company's security policy states that nobody in any member account, including account administrators and the root user, may stop or delete CloudTrail trails.
>
> The company uses AWS Organizations with all features enabled.

**What should the DevOps engineer implement?**

A. A permissions boundary on every IAM role in every account

B. An IAM policy denying the actions, attached to every user and role

C. A session policy applied whenever administrators assume roles

D. An SCP denying cloudtrail:StopLogging and cloudtrail:DeleteTrail, attached to the root

◆ PAUSE AND DECIDE

## What should the DevOps engineer implement?

A **Boundaries everywhere**

*Administrators could remove them, and boundaries do not apply to the root user.*

B **IAM deny policies**

*Account administrators can detach them, and they miss the root user.*

C **Session policies**

*Only apply when someone chooses to assume a role with one.*

D **SCP at the root**

Applies to everyone in member accounts, root user included, and cannot be changed from inside.

THE PAIR

Axis 2: a ceiling nobody inside the account can lift, root included, is an SCP.

> The security team must be notified within seconds every time anyone creates an IAM access key in the production account.
>
> A CloudTrail trail is already logging management events.

**What should the DevOps engineer set up?**

A. An EventBridge rule on the CreateAccessKey API call, targeting an SNS topic

B. A periodic AWS Config rule checking for access keys

C. A Security Hub CSPM control for IAM access keys

D. IAM Access Analyzer unused access findings

◆ PAUSE AND DECIDE

## What should the DevOps engineer set up?

**A** EventBridge on the API call

Fires on each call within seconds, through the existing trail.

**B** Periodic Config rule

*Evaluates state on a schedule, perhaps hours later.*

**C** CSPM control

*Scores posture over time. Not a per-event alert.*

**D** Unused access

*Finds keys that are not used. The opposite question.*

THE PAIR

Axis 1 again: something just happened means EventBridge; is it out of policy means Config.

> An application uses Aurora PostgreSQL with a single writer instance. It must survive the loss of an Availability Zone, failing over automatically within minutes.
>
> The company has no multi-Region requirement and wants the lowest cost.

**What should the DevOps engineer do?**

A Convert the cluster to an Aurora Global Database

B Add a cross-Region read replica

C Add an Aurora replica in a different Availability Zone

D Migrate the data to DynamoDB global tables

◆ PAUSE AND DECIDE

## What should the DevOps engineer do?

A Global Database

*Protects against Region loss, at higher cost. No such requirement.*

B Cross-Region replica

*Another Region, not another zone, and promotion is manual.*

C Replica in another AZ

Aurora promotes it automatically if the writer's zone fails. Cheapest fit.

D DynamoDB global tables

*A re-platform for a requirement one replica meets.*

THE PAIR

Axis 6: match the scope of protection to the scope of failure. Zone, not Region.

> An EC2 instance in a production account has started communicating with an IP address associated with cryptocurrency mining.
>
> The security team wants such activity detected and sent to its incident workflow automatically.

## Which service detects this?

A Amazon Inspector

B Amazon Macie

C Security Hub CSPM controls

D Amazon GuardDuty, with findings routed through EventBridge

◆ PAUSE AND DECIDE

## Which service detects this?

A Inspector

*Finds vulnerable software, not active malicious behaviour.*

B Macie

*Reads S3 object contents. Unrelated to network behaviour.*

C CSPM controls

*Evaluates configuration against standards, not traffic.*

D GuardDuty

Detects threats from flow and DNS activity, including mining, and emits EventBridge events.

THE PAIR

Axis 3: active threat means GuardDuty. A weakness that could be exploited means Inspector.

> Checkout latency tripled for an hour after an unremarkable afternoon. There is no sign of attack. Engineers want help correlating metrics, logs, deployments and configuration changes into likely causes.
>
> Both Detective and CloudWatch are available.

**What should the engineers use?**

A Amazon Detective

B A CloudWatch investigation

C Amazon GuardDuty

D Security Hub

◆ PAUSE AND DECIDE

## What should the engineers use?

A Detective

*Scopes security incidents. It does not analyse latency or deployments.*

B CloudWatch investigation

Correlates operational signals, including deployments and changes, into hypotheses.

C GuardDuty

*Detects threats. There is no attack here.*

D Security Hub

*Security findings and posture, not operational performance.*

THE PAIR

Axis 3: an operational problem goes to CloudWatch investigations; a security incident goes to Detective.

> A service reads about 200 configuration values at start-up, such as endpoints, timeouts and feature settings. None are secrets, and none need rotation or gradual rollout.
>
> The team wants hierarchical names per environment, and the lowest cost.

## Where should the values be stored?

A AWS Secrets Manager, one secret per value

B A DynamoDB table read at start-up

C Parameter Store standard parameters, with paths such as /prod/checkout/timeout

D AWS AppConfig with a linear deployment strategy

◆ PAUSE AND DECIDE

## Where should the values be stored?

A Secrets Manager

*Charged per secret, for rotation nobody needs: 200 monthly charges.*

B DynamoDB

*Works, but it is a table to design and pay for, with no hierarchy built in.*

C Parameter Store standard

Free, hierarchical by path, ideal for configuration read at start-up.

D AppConfig linear

*Built for gradual, validated rollout, which is explicitly not needed.*

THE PAIR

Axis 8: rotation means Secrets Manager; gradual rollout means AppConfig; plain configuration means Parameter Store.

> A Java API on Lambda shows high p99 latency after idle periods, caused by long cold starts while the framework initialises.
>
> The team wants to cut cold-start time at the lowest cost, without changing application code.

**What should the DevOps engineer do?**

A Enable SnapStart on the function's published versions

B Set reserved concurrency on the function

C Configure provisioned concurrency sized for peak traffic

D Increase the function's memory to the maximum

◆ PAUSE AND DECIDE

## What should the DevOps engineer do?

**A** SnapStart

Restores from a snapshot taken after initialisation. For Java it adds no charge.

**B** Reserved concurrency

*Controls how much, not how ready. Cold starts continue.*

**C** Provisioned concurrency

*Removes cold starts, but charges for warm environments all the time.*

**D** Maximum memory

*Somewhat faster initialisation, at a higher price per invocation.*

THE PAIR

Axis 6: how ready, cheaply, for Java: SnapStart. How ready, whatever the cost: provisioned concurrency.

# Module 28 in five lines

- Two options both sound right? Find the axis they differ on.
- Before or after; who is acting; contents or behaviour; copy or look.
- Once or continuously; how much is running; where the failure lands; which layer.
- Then ask that one question of the scenario. The planted detail usually answers it.
- Match the scope of the answer to the scope of the problem.

> ON THE EXAM
>
> Next, module 29: how DOP-C02 questions are written, and how to read them under time pressure.